

Day 8: JS Introduction, Variables & Operators

Foundational language concepts, memory scopes, data type classification, and strict operators.

1. Execution Environment & Variable Declarations

JavaScript is a single-threaded, dynamically typed scripting language executing inside browser JS engines (V8, SpiderMonkey). Modern code uses `let` and `const` for block scoping.

Keyword	Scope	Re-assignment	Re-declaration	Best Practice
<code>var</code>	Function	Allowed	Allowed	Avoid (legacy)
<code>let</code>	Block <code>{ }</code>	Allowed	Denied	Use when value mutates
<code>const</code>	Block <code>{ }</code>	Denied	Denied	Default Choice

2. Data Type Classification

Primitive Types (Passed by Value)

- `String` : Text data e.g. `"Monstera"`
- `Number` : Integers & floats e.g. `25` , `3.14`
- `Boolean` : Logical `true` or `false`
- `Null` : Explicit empty/absence of value
- `Undefined` : Uninitialized variable

Reference Types (Passed by Memory Reference)

- `Object` : Key-value collections `{ name: "Fern" }`
- `Array` : Ordered lists `[1, 2, 3]`
- `Function` : Callable code blocks

3. Strict Comparison & Operators

Always prefer strict equality (`===`) to prevent unexpected type coercion errors caused by loose equality (`==`).

```
// Day 8 Basic Exercises: Operators & Types
const plantName = "Peace Lily"; let
moisturePercent = 28; const minThreshold = 30; // Logical expression evaluation
const needsWater = moisturePercent < minThreshold; // true
// Strict vs Loose Equality Comparison
console.log(10 == "10"); // true (Coerces string "10" to number)
console.log(10 === "10"); // false (Strict check: Number vs String)
```

Day 8 Key Takeaway

Declare variables with `const` by default. Only switch to `let` if you explicitly need to reassign the variable value later in your code execution flow.

Day 9: Conditions, Loops, Functions & Assignment 6

Control flow structures, arrow functions, array iteration methods, and plant data processing.

1. Control Flow & Function Syntax

Ternary & Conditional Logic

```
const status = (moisture < 30) ?  
  "Water Immediately" : "Moisture Adequate";
```

Arrow Functions

```
const calculateCareDays = (lastDays,  
  cycle) => { return cycle - lastDays; };
```

2. Essential Array Methods Cheatsheet

Method	Purpose	Return Value	Example
<code>.filter()</code>	Selects items matching a predicate function	New Array (filtered)	<code>plants.filter(p => p.isThirsty)</code>
<code>.find()</code>	Finds first element matching criteria	Single Element / undefined	<code>plants.find(p => p.id === 1)</code>
<code>.map()</code>	Transforms each element in an array	New Array (transformed)	<code>plants.map(p => p.name)</code>
<code>.forEach()</code>	Executes callback for each element	undefined	<code>plants.forEach(p => render(p))</code>

3. Assignment 6 Implementation: Plant-Data Program

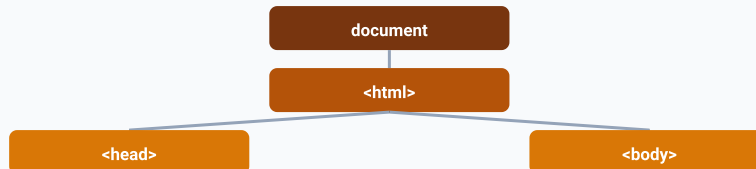
```
// Assignment 6: Data Program Managing Plant Objects  
const plantDatabase = [ { id: 1, name: "Monstera Deliciosa", category: "Indoor", waterDays: 7, light: "Bright Indirect" }, { id: 2, name: "Snake Plant", category: "Low Light", waterDays: 14, light: "Low to Bright" }, { id: 3, name: "Peace Lily", category: "Indoor", waterDays: 4, light: "Medium" }, { id: 4, name: "Echeveria Succulent", category: "Succulents", waterDays: 10, light: "Direct Sun" } ];  
  
// 1. Filter function: Returns plants requiring watering within threshold  
const getThirstyPlants = (maxDays) => { return plantDatabase.filter(plant => plant.waterDays <= maxDays); };  
  
// 2. Search function: Case-insensitive plant search by name string  
const searchPlantByName = (query) => { return plantDatabase.find(plant => plant.name.toLowerCase().includes(query.toLowerCase())); };  
  
// Console Output Verifications  
console.log("Plants needing water <= 5 days:", getThirstyPlants(5));  
console.log("Search result for 'snake':", searchPlantByName("snake"));
```

Day 10: DOM Selection & Dynamic Content

Document Object Model architecture, element query selection, content manipulation, and dynamic HTML card rendering.

1. Document Object Model (DOM) Tree Structure

The browser parses HTML markup into a tree of node objects that JavaScript can inspect, modify, add, or delete in real time.



2. Element Selection Methods

Method	Selector Type	Return Value	Common Usage
<code>document.getElementById()</code>	Unique ID string	Single Element / <code>null</code>	Targeting unique containers
<code>document.querySelector()</code>	CSS Selector string	First Match Element	Targeting by class or tag
<code>document.querySelectorAll()</code>	CSS Selector string	Static <code>NodeList</code>	Iterating over element groups

3. Content & Styling Manipulation

Updating Text vs. Inner HTML

Use `textContent` for plain text (prevents XSS attacks).
Use `innerHTML` only when injecting HTML elements.

CSS Class Management

Use `classList.add('active')`, `.remove('hidden')`, or `.toggle('selected')` rather than inline styles.

4. Code Blueprint: Dynamic Plant Card Rendering

```
const gridContainer = document.getElementById("plantGrid"); // Render plant array objects
directly into the DOM as HTML cards
function displayPlants(plantsList) {
  gridContainer.innerHTML = ""; // Clear previous content
  plantsList.forEach(plant => {
    const cardHTML = `<div class="col-md-4">
      <div class="card p-3 border-warning shadow-sm">
        <h5 class="text-warning">${plant.name}</h5>
        <p class="text-muted">Category: ${plant.category}</p>
        <p>Water every <strong>${plant.waterDays}</strong> days</p>
      </div>
    </div>`;
    gridContainer.innerHTML += cardHTML;
  });
}
```

Day 11: Events, Dropdowns & Form Handling

Interactive event listeners, form submit intercept, dropdown filtering, and real-time state synchronization.

1. Event Listener Architecture

Events respond to user interactions (clicks, keypresses, select changes, form submissions). Use `addEventListener(type, handler)` to decouple event logic from HTML markup.

Event Name	Target Element	Trigger & Primary Use Case
click	Buttons, Icons, Cards	User clicks or taps element. Triggers modal/action.
input	Text Inputs, Search Bars	Triggers instantly on every keystroke for real-time search.
change	Select Dropdowns, Checkboxes	Triggers when option selection value is finalized.
submit	<form> Containers	Triggers on form submit. Requires <code>e.preventDefault()</code> .

2. Complete Interactive Application Script

Connecting search input, category dropdown, and form inputs to dynamic DOM rendering:

```
// Day 11 Interactive App Script
const searchInput = document.getElementById("searchInput");
const categorySelect = document.getElementById("categorySelect");
const addPlantForm = document.getElementById("addPlantForm");
// Combined Filter Function (Handles both Search and Dropdown Filter)
function handleFilter() {
  const query = searchInput.value.toLowerCase();
  const selectedCat = categorySelect.value;
  const filtered = plantDatabase.filter(plant => {
    const matchesSearch = plant.name.toLowerCase().includes(query);
    const matchesCategory = selectedCat === "All" || plant.category === selectedCat;
    return matchesSearch && matchesCategory;
  });
  displayPlants(filtered); // Re-renders UI Grid
} // Event Attachments
searchInput.addEventListener("input", handleFilter);
categorySelect.addEventListener("change", handleFilter);
// Form Submit Handler: Adds new plant to array without page reload
addPlantForm.addEventListener("submit", function(e) {
  e.preventDefault(); // Prevent browser refresh
  const newPlant = {
    id: plantDatabase.length + 1,
    name: document.getElementById("nameInput").value,
    category: document.getElementById("catInput").value,
    waterDays: Number(document.getElementById("waterInput").value),
    light: "Medium"
  };
  plantDatabase.push(newPlant);
  displayPlants(plantDatabase); // Update DOM view
  addPlantForm.reset(); // Clear inputs
});
```

Module 2 Synthesis

By combining array state (Data), DOM manipulation (View), and Event Listeners (Control), you have built a complete Single Page Application (SPA) architecture in vanilla JavaScript.